

COMS W4156 Advanced Software Engineering (ASE)

December 7, 2023



Agenda

1. test oracles
2. pseudo-oracles
3. if time permits: test generation



What is a Test Oracle?

An entity that already knows the expected outputs and/or can check that the actual outputs are the same as or consistent with the expected outputs



Often the human tester is the test oracle - either they know the correct output in advance or, given an actual output, can determine whether or not it is correct

For manual testing, the human tester checks the results

For automated testing, the human tester writes assertions to automatically check the results

Test Assertions

Notation for checking the results of executing a test

Usually supports (at least) checking for true/false result, comparing an output to a known value, and checking whether an exception has been raised

[Google Test assertions](#)

[JUnit 5 Assertions](#)

[Jest assertions](#)

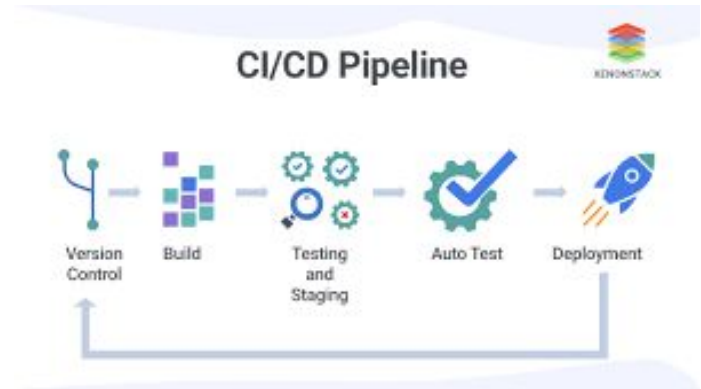


Why Are Good Test Assertions Important?

Continuous Integration (CI) tools hook a version control system with a build tool to automatically re-run the build after every commit to the shared repository, to detect errors quickly

Continuous Delivery/Deployment (CD) adds automatic delivery and/or deployment to continuous integration = produces a patch for customers and/or installs new build on the production server(s)

Continuous deployment (and possibly continuous delivery) has no human in the loop!



CD requires that testers be able to use automated testing tools, with automatically checked assertions, for all testing, including end-to-end system testing and acceptance testing - not just for unit and integration testing

How Does the Tester Know the Correct Answer to Assert?

If it doesn't crash, hang, or produce obviously nonsensical output, it works!

Stick to math and string processing libraries, with well-understood functions

Wait until users complain that the software doesn't do what they expected, then write test cases to check for what they say they expected



Prose Specifications

Detailed prose specification - common for networking, database, file system and other [standard protocols](#) where interoperability is a primary concern, rare for user-facing functionality of business/consumer applications

[\[Search\]](#) [\[txt\]](#) [\[html\]](#) [\[pdf\]](#) [\[with errata\]](#) [\[bibtex\]](#) [\[Tracker\]](#) [\[WG\]](#) [\[Email\]](#) [\[Diff1\]](#) [\[Diff2\]](#) [\[Nits\]](#)

From: [draft-ietf-uri-ur1-07](#)

Obsoleted by: [4248](#), [4266](#)

Updated by: [1808](#), [2368](#), [2396](#), [3986](#), [6196](#), [6270](#),
[8089](#)

Network Working Group

Request for Comments: 1738

Category: Standards Track

Proposed Standard

[Errata exist](#)

T. Berners-Lee

CERN

L. Masinter

Xerox Corporation

M. McCahill

University of Minnesota

Editors

December 1994

Uniform Resource Locators (URL)

Status of this Memo

This document specifies an Internet standards track protocol for the Internet community, and requests discussion and suggestions for improvements. Please refer to the current edition of the "Internet Official Protocol Standards" (STD 1) for the standardization state and status of this protocol. Distribution of this memo is unlimited.

Abstract

This document specifies a Uniform Resource Locator (URL), the syntax and semantics of formalized information for location and access of resources via the Internet.

1. Introduction

This document describes the syntax and semantics for a compact string representation for a resource available via the Internet. These strings are called "Uniform Resource Locators" (URLs).

The specification is derived from concepts introduced by the World-Wide Web global information initiative, whose use of such objects dates from 1990 and is described in "Universal Resource Identifiers in WWW", [RFC 1630](#). The specification of URLs is designed to meet the requirements laid out in "Functional Requirements for Internet Resource Locators" [\[12\]](#).

This document was written by the URI working group of the Internet Engineering Task Force. Comments may be addressed to the editors, or to the URI-WG uri@bunyip.com. Discussions of the group are archived at [URL:http://www.acl.lanl.gov/URI/archive/uri-archive.index.html](http://www.acl.lanl.gov/URI/archive/uri-archive.index.html)

Formal Specifications



Other than standard protocols and safety-critical systems, 'specifications' are usually informal user stories and/or use cases that leave test oracles up to developers - this is ok for most software

Formal Specification - expensive to write and verify, mostly limited to safety-critical systems

$$\frac{P \rightarrow P' \quad \{P'\} C \{Q'\} \quad Q \rightarrow Q'}{\{P\} C \{Q\}}_{\text{WEAK}}$$

$$\frac{}{\{A[E/x]\} x := E \{A\}}_{\text{ASG}}$$

$$\frac{\{P\} C_1 \{A\} \quad \{A\} C_2 \{Q\}}{\{P\} C_1; C_2 \{Q\}}_{\text{SEQ}}$$

$$\frac{\{P \wedge B\} C_1 \{Q\} \quad \{P \wedge \neg B\} C_2 \{Q\}}{\{P\} \text{ if } B \text{ then } C_1 \text{ else } C_2 \{Q\}}_{\text{IF}}$$

$$\frac{\{I \wedge B\} C \{I\}}{\{I\} \text{ while } B C \{I \wedge \neg B\}}_{\text{While}}$$

Agenda

1. test oracles
2. pseudo-oracles
3. if time permits: test generation



Sometimes There Isn't An Agreed Upon Test Oracle

Consider this search: <https://www.google.com/search?q=gail+kaiser>

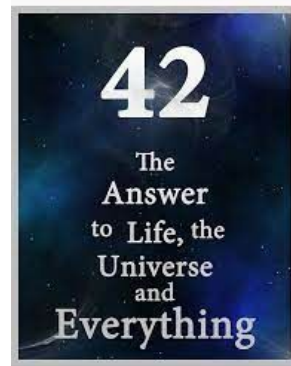
Is the answer correct?

Are the answers on the first page the best ones?

Now consider this search: [gail kaiser - Search \(bing.com\)](https://www.bing.com/search?q=gail+kaiser)

Does it give the same links in the same order?

[Google](#) and [Microsoft](#) put a lot of time and effort into rigorously testing their search engines, but their search engine results do not always agree



Sometimes There Isn't Any Test Oracle At All

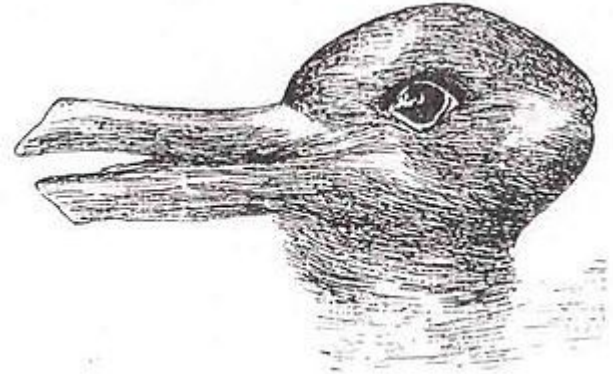
Write a program (using a conventional algorithm, data mining, machine learning, or any other technique) that always predicts correctly what Gail Kaiser will eat for breakfast the next day

Write a program that determines exactly how many stars there are in the universe

Write a program that solves the Halting Problem

Write a program that correctly labels ambiguous images

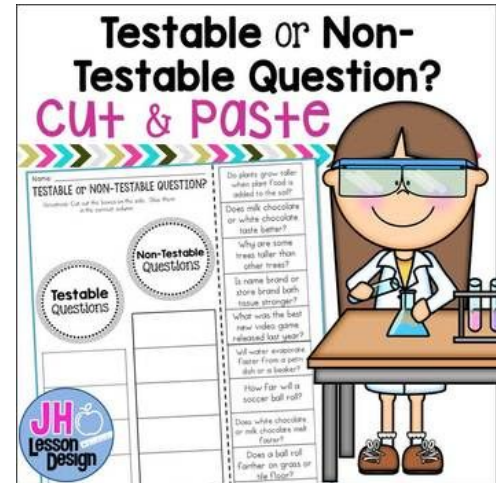
Write a program that calculates when the Earth will run out of fossil fuels



Non-Testable Programs

Non-testable program = *“Programs which were written in order to determine the answer in the first place. There would be no need to write such programs, if the correct answer were known.”* ([Weyuker 1982](#))

? Which of the previous kinds of examples does this quote intend to address ?

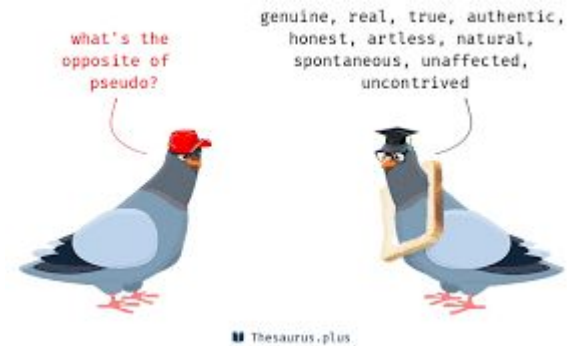


Pseudo-Oracles

Pseudo-oracle = one test case acts as an oracle for another test case

You may not know whether the result of either test case is right, but if the results are inconsistent you know (at least) one of them is wrong

They could both be wrong



Pseudo-Oracles: Regression Testing

[Regression testing](#) is the most common form of pseudo-oracle

The test results for the old version are known and we re-run the tests on the new version to see if they have the same or different results

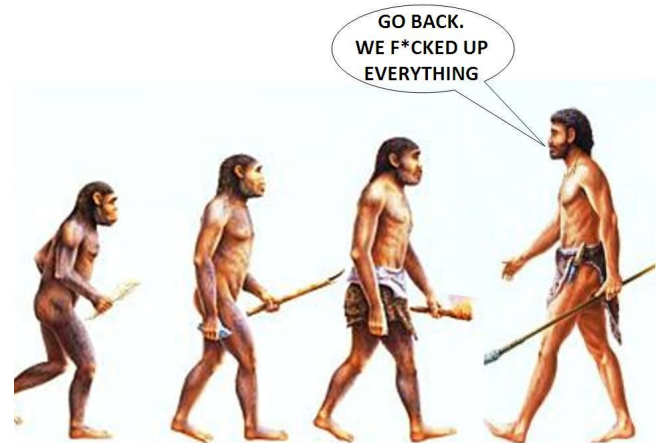
Some test cases *should* change results on the new version $\Rightarrow$ when we try to fix a bug that caused a test case to fail on the old version, we want that test case to pass on the new version

But unrelated test cases should not be affected $\Rightarrow$ if they are, that indicates a hidden dependency and (probably) a newly introduced bug

```
program_v1(input) -> output_v1
```

```
program_v2(input) -> output_v2
```

```
output_v1 =? output_v2
```



Pseudo-Oracles: Differential Testing

Differential testing requires a second *independently developed* implementation of the same functionality, as well as a test inputs applicable to both

Practical for standard protocols, e.g., there are many different implementations of SSL/TLS and HTTP

- If they disagree, one or both is wrong
- If they agree, that does not guarantee both are right, but does provide some confidence that they are both right

? What other kinds of software typically have two or more *independent* implementations ?

```
program_1(input) -> output_1
```

```
program_2(input) -> output_2
```

```
output_1 ==? output_2
```



It's critical that the implementations are truly independent, with no common third-party libraries or other common factors. ? Why ?

But “independent” developers can make the same mistakes

Pseudo-Oracles: Metamorphic Testing

[Metamorphic testing](#) requires only one implementation

Starts with some original input and its known original output. We do not need to know whether this output is correct

Create a new test case by *deriving* a new input from the original input, and *predicting* the expected new output from the original input, the derived input and the original output - the prediction is usually that the output is the same, nearly the same, or changed in a simple way

If the actual new output deviates too much from the predicted output, there is (probably) a bug

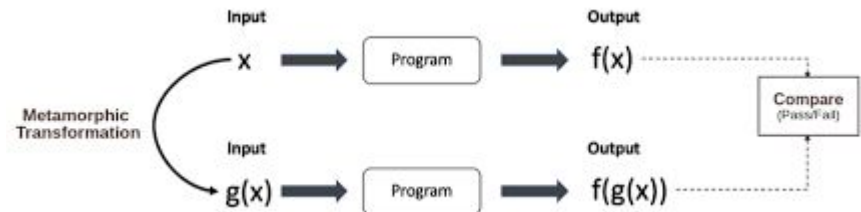
```
program(input_1) -> output_1
```

```
deriveInput(input_1) -> input_2
```

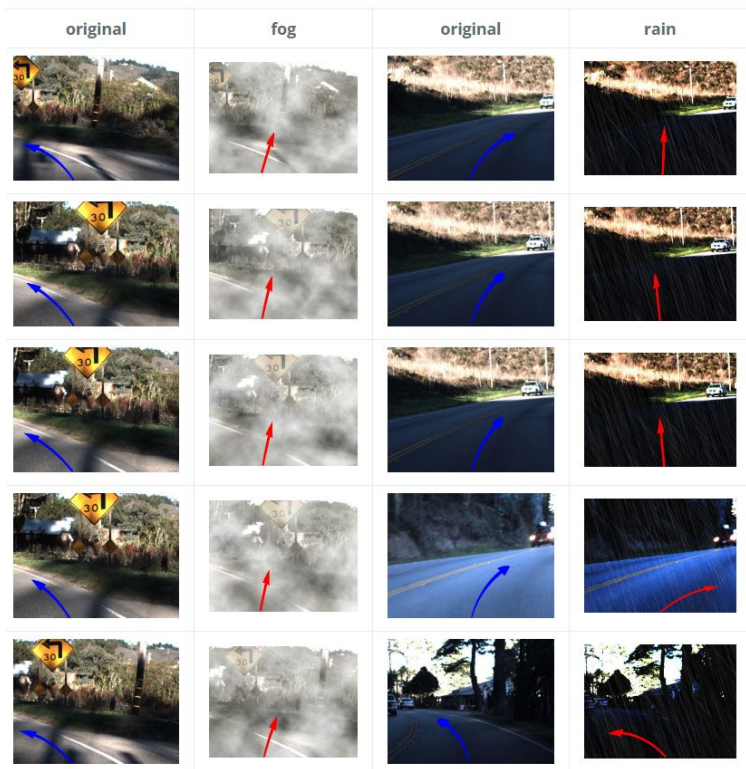
```
predictOutput(input_1, input_2, output_1) -> predicted_output_2
```

```
program(input_2) -> output_2
```

```
check(predicted_output_2, output_2)  
-> do they match?
```



Example Metamorphic Testing



Say we have a machine learning application that controls the steering of a [self-driving car](#), so it obeys traffic laws and doesn't cause accidents

How do we know what is “correct” steering for a given real-world driving scenario? How can we write test cases?

Different human drivers will not necessarily steer at exactly the same angle and the same human driver will not necessarily steer at the same angle every time

[DeepTest](#): Synthetic but realistic changes to driving images such as adding snow, fog, rain, more sunlight, less sunlight, etc. should not (in most cases) change automobile steering

Predicted Output is not Necessarily the Same Output

Metamorphic relations for a sorting program (lowest first)
applied to an array of numbers
`sort(5,2,3,1,4) -> 1,2,3,4,5`

Permutative: if we shuffle the order of the input array, the
sorted output array should be the same
`sort(3,4,1,2,5) -> 1,2,3,4,5` 😊

Additive: if we add N to every element of the input array,
the sorted output array should be in the same order
`sort(25,22,23,21,24) -> 21,22,23,24,25` 😊

Multiplicative: if we multiply every element of the input
array by (positive) N, the sorted output array should be
in the same order
`sort(15,6,9,3,12) -> 3,6,9,12,15` 😊

Invertive: if we multiply every element of the input array
by (negative) N, the sorted output array should be in the
opposite order
`sort(-5,-2,-3,-1,-4) -> -5,-4,-3,-2,-1` 😊

Inclusive: if we add one new element to the input array,
the sorted output array should be the same except for
the placement of that new element
`sort(5,2,42,3,1,4) -> 1,2,3,4,5,42` 😊

Exclusive: if we remove one element from the input
array, the sorted output array should be the same except
it is missing that one element
`sort(5,3,1,4) -> 1,3,4,5` 😊

Using Metamorphic Relations to Detect Bugs

Original

`buggysort(5,2,3,1,4) -> 1,2,3,4,5`

Permutative: if we shuffle the order of the input array, the sorted output array should be the same

`buggysort(3,4,1,2,5) -> 2,4,1,5,3` 😞

Additive: if we add N to every element of the input array, the sorted output array should be in the same order

`buggysort(25,22,23,21,24) -> 25,25,25,25,25` 😬

Multiplicative: if we multiply every element of the input array by (positive) N, the sorted output array should be in the same order

`buggysort(15,6,9,3,12) -> 6,12,3,15,9` 😞

Invertive: if we multiply every element of the input array by (negative) N, the sorted output array should be in the opposite order

`buggysort(-5,-2,-3,-1,-4) -> -1,-4,-3,-2,-5` 😞

Inclusive: if we add one new element to the input array, the sorted output array should be the same except for the placement of that new element

`buggysort(5,2,42,3,1,4) ->` ⌚ 😬

Exclusive: if we remove one element from the input array, the sorted output array should be the same except it is missing that one element

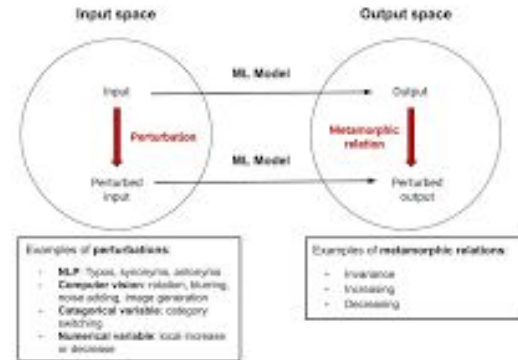
`buggysort(5,3,1,4) ->` 💣 😬

Where Do Metamorphic Relations Come From?

Some are generic, e.g., permutation can be applied to any ordered input

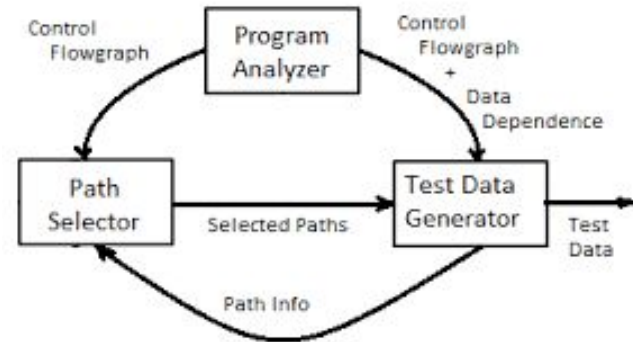
Others are specific to domain or application such as car steering

Metamorphic Testing automates generation of additional tests from existing tests whether or not there is a test oracle - it's a test generation technique as well as a pseudo-oracle technique



Agenda

1. test oracles
2. pseudo-oracles
3. if time permits: test generation



Fuzzing (aka Fuzz Testing)

Probably best known "test generation" technique

Start with a “seed” input and semi-randomly mutate it (or parts of it), intending to construct malformed or invalid input

Normally applied to relatively long inputs or sequences of inputs like packets, files, and protocols, not to primitive types like integers and strings except as they appear in the fields of larger structured content

Fuzzer leverages mandatory features and constraints of data types and metadata formats, e.g., required fields set to zero length, metadata says the contents are version 3 format but the contents are version 1 and incompatible with version 3



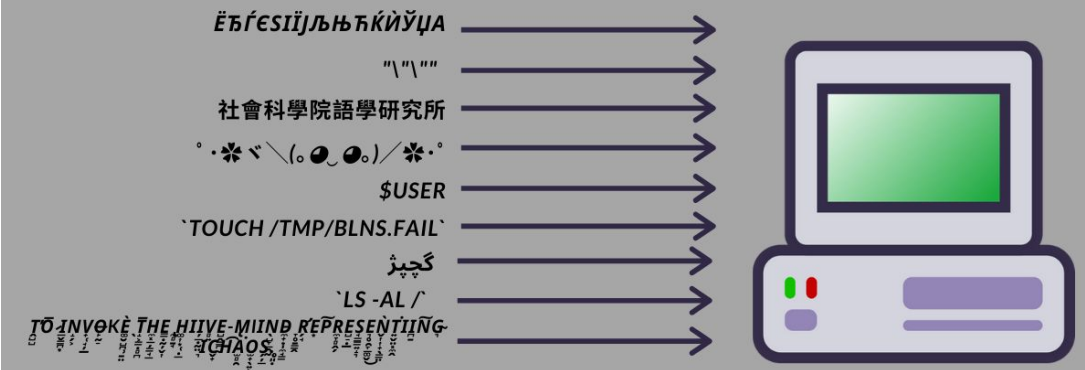
Examples

Power للصبَّالصبَّرَ ٩ ٩h ٩ ٩٧

The Neẓperdian hive-mind of chaos
Zalgo



WHAT IS FUZZING?



Fuzzers usually do not know anything about program semantics, so assertions limited to program crashes, program is very slow, program uses too much memory, etc.

Fuzzing



Fuzz Testing is mostly used to detect security vulnerabilities, so generation tries to produce illegal inputs that make it past any input checkers, validators or sanitizers that typically operate at entry points. *Unchecked* invalid input is how hackers get past defenses and trick software into processing their specially crafted data (“attack vector”)

Feedback/guidance encourages generating new inputs that exercise longer execution paths that reach further into the target software - e.g., after the fuzzer finds a malformed packet header that is accepted by the program, it keeps using that header while fuzzing different parts of the packet body

Like random test generation, also easy to implement and use, and yields lots of test inputs - but requires some existing tests as seeds

examples: [AFL++](#), [OSS-Fuzz](#)

Free Fuzzing Book

[The Fuzzing Book](#)

From the same lead author as free [The Debugging Book](#)



Automated Test Generation

Metamorphic testing generates both the test input and the assertion (“is it consistent with the predicted output?”) - although it requires some existing tests to derive from and thus is limited by the quality of the original test suite

Regression, differential and fuzz testing also require an existing test suite

But ideally test generation tools could augment even poor quality or non-existent manually written test cases with high quality test suites



Most Test Generators are Input Generators

The best most test generation techniques can do is generate inputs that are useful for triggering bugs, which is hard, but generating high quality assertions that can accurately detect triggered bugs can be even harder

The assertions typically must be written by human developers OR easy to check generic assertions are used, like did it store the input value, did it crash?

Some test generators assume the current code is correct (🧐), but bugs may be inserted later, so the results of testing the current code are recorded and those results are inserted into the assertions used during regression testing



Random Test Generation

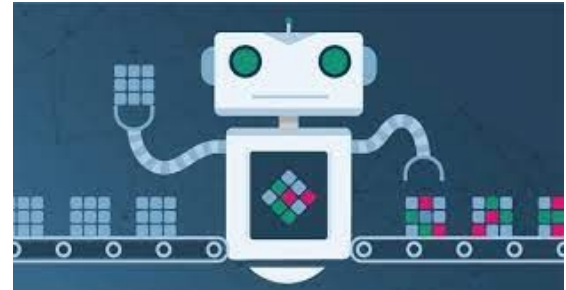
Selects inputs at random from the input space based on primitive type, e.g., int

The test generator need not know anything about equivalence partitioning except for built-in partitions of primitive types, e.g., MinInt, negative, zero, positive, MaxInt

Possible lengths of strings and arrays might be determined by conventional data sizes: empty, one element, four elements, eight elements, 16, 32, 64, 128, 256, etc. elements (stop growing when code under test starts failing)

Easy to implement and use, yields lots of test inputs, but not necessarily a "high quality test suite"

example: [Randoop](#)



Example

```
int myAbs(int x) {  
    if (x > 0) {  
        return x;  
    }  
    else {  
        return x; // bug: should be '-x'  
    }  
}
```

The random tests for this function could be {123, 36, -35, 48, 0}. Of these, only the value '-35' triggers the bug

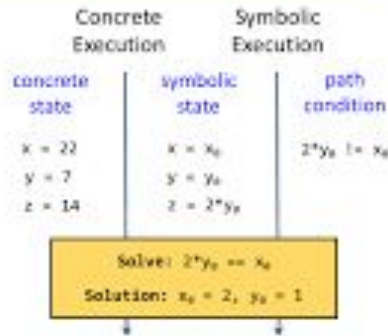
An assertion could be added to check the results, but how does the test generator know what this assertion should be?

```
void testAbs(int n) {  
    for (int i=0; i<n; i++) {  
        int x = getRandomInput();  
        int result = myAbs(x);  
        assert(result >= 0);  
    }  
}
```

Symbolic Execution

An Illustrative Example

```
int foo(int v) {  
    return 2*v;  
}  
  
void test_me(int x, int y) {  
    int z = foo(y);  
    if (z == x)  
        if (x > y+10)  
            ERROR;  
}
```



Generalizes testing by executing the program abstractly, so one abstract execution covers multiple possible inputs of the program that share a particular execution path through the code

Useful for increasing coverage: execute symbolically from a program entry point until execution reaches an uncovered branch, then use constraint solving on the path condition to identify at least one set of inputs that will force executing that path. Construct a conventional test case with those inputs

If the path demonstrates an error (logical contradiction), then such an input is called a “witness”

Example where g is path condition and E is symbolic environment

1 int x=0, y=0, z=0;

2 if(a) {

3 x = -2;

4 }

5 if (b < 5) {

6 if (!a && c) { y = 1; }

7 z = 2;

8 }

9 assert(x + y + z != 3);

line	g	E
0	true	$a \mapsto \alpha, b \mapsto \beta, c \mapsto \gamma$
1	true	$\dots, x \mapsto 0, y \mapsto 0, z \mapsto 0$
2	$\neg \alpha$	$\dots, x \mapsto 0, y \mapsto 0, z \mapsto 0$
5	$\neg \alpha \wedge \beta \geq 5$	$\dots, x \mapsto 0, y \mapsto 0, z \mapsto 0$
9	$\neg \alpha \wedge \beta \geq 5 \wedge 0 + 0 + 0 \neq 3$	$\dots, x \mapsto 0, y \mapsto 0, z \mapsto 0$

line	g	E
0	true	$a \mapsto \alpha, b \mapsto \beta, c \mapsto \gamma$
1	true	$\dots, x \mapsto 0, y \mapsto 0, z \mapsto 0$
2	$\neg \alpha$	$\dots, x \mapsto 0, y \mapsto 0, z \mapsto 0$
5	$\neg \alpha \wedge \beta < 5$	$\dots, x \mapsto 0, y \mapsto 0, z \mapsto 0$
6	$\neg \alpha \wedge \beta < 5 \wedge \gamma$	$\dots, x \mapsto 0, y \mapsto 1, z \mapsto 0$
6	$\neg \alpha \wedge \beta < 5 \wedge \gamma$	$\dots, x \mapsto 0, y \mapsto 1, z \mapsto 2$
9	$\neg \alpha \wedge \beta < 5 \wedge \neg(0 + 1 + 2 \neq 3)$	$\dots, x \mapsto 0, y \mapsto 1, z \mapsto 2$

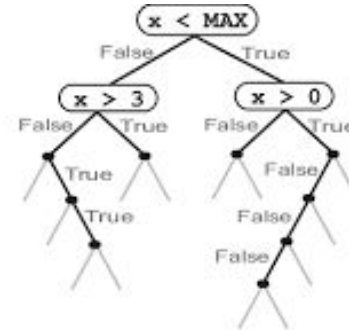
Concolic Execution

“Concolic execution” is a special case of symbolic execution where some of the inputs are concrete (specific known values) and some are symbolic (α , β , γ , `$thisIsAnInteger`)

Symbolic and concolic execution are not easy to implement and use, do not scale to large programs due to path explosion

example: [Klee](#), [JavaPathFinder](#)

```
void write( int x )  
{  
  if (x < MAX) {  
    if (x > 0)  
    ...  
  } else {  
    if (x > 3)  
    ...  
  }  
}
```



Remaining Assignments

→ [Second Iteration Demo](#) due tonight by 11:59pm

Second Assessment will be available later tonight, Friday December 8 at 12:01am, and due Monday December 11 at 11:59pm (4 days minus two minutes)

[Demo Day](#) is Monday, December 18, 10am-4pm, signup sheet is [here](#)



Ask Me Anything

